

NomadNest

Rapport Technique

**Plateforme de co-working et mise en relation de nomades
digitaux**

Projet _Technologies Web 1 Préparatoire intégré première année B _2026

HTML & CSS · JavaScript · PHP · MySQL



Réalisé par:

Hakiri Salma

Chaari Ines

Jemai Farah

Projet : Plateforme web de co-working — NomadNest

Technologies : HTML5, CSS3, JavaScript (ES6), PHP 8, MySQL 8

Environnement de développement : Laragon, VS Code, HeidiSQL, Git / GitHub

Année universitaire : 2025 – 2026

1. Présentation du thème et des objectifs

NomadNest est une plateforme web de mise en relation entre travailleurs nomades et hôtes proposant des espaces de co-working. Dans un contexte où le télétravail et les modes de vie nomades se généralisent, l'accès à des espaces de travail flexibles et à une communauté professionnelle locale devient un besoin réel.

L'objectif principal du projet est de concevoir un site web complet et fonctionnel permettant de :

- Rechercher et réserver des espaces de co-working filtrés par ville, type et équipements
- Consulter les profils des membres et développer un réseau professionnel
- Gérer ses réservations via un tableau de bord personnel
- Permettre aux hôtes de publier et administrer leurs espaces
- Communiquer par messagerie interne entre membres et hôtes

Le site est créé avec des technologies web standard : HTML, CSS, JavaScript, PHP et MySQL.

2. Architecture du projet

2.1 Structure des fichiers

Le projet est organisé selon une séparation claire des responsabilités :

Dossier/Fichier	Role
/css/style.css	Feuille de style globale (variables, composants, responsive)
/js/*.js	Scripts JavaScript par page (spaces.js, auth.js, dashboard.js...)
/pages/*.php	Pages PHP accessibles à l'utilisateur (spaces, auth, dashboard...)
/api/*.php	Endpoints backend retournant du JSON (consommés par le JS)
/includes/db.php	Connexion PDO à la base de données (partagée par toutes les pages)
/includes/auth_check.php	Helpers de session : require_login(), require_host()
/includes/navbar.php	Barre de navigation incluse sur toutes les pages
/includes/footer.php	Pied de page inclus sur toutes les pages
/index.php	Page d'accueil (landing page)
/nomadnest.sql	Schéma complet de la base de données + données de test
/docs/rapport_technique.pdf	Ce rapport

2.2 Schéma de la base de données

La base de données **Nomadnest** contient 8 tables reliées par des clés étrangères.

Les

suppressions en cascade garantissent l'intégrité référentielle.

Table	Colonnes Principales	Description
users	id, name, email, password, role, city, bio, tags, status	Comptes membres et hôtes
spaces	id, host_id, name, type, city, price_per_day, image, rating	Espaces de co-working
amenities	id, space_id, name	Équipements par espace

bookings	id, user_id, space_id, date_start, date_end, seats,status	Réservations
reviews	id, user_id, space_id, rating, comment	Avis et notes
messages	id, sender_id, receiver_id, body, is_read	Messagerie entre membres
subscriptions	id, user_id, plan, price, active	Abonnements (free/pro/business)
connections	id, from_user, to_user, status	Réseau entre membres

Relations principales : spaces.host_id → users.id | bookings.user_id → users.id | bookings.space_id → spaces.id | reviews.(user_id, space_id) → (users, spaces) | messages.(sender_id, receiver_id) → users.id | subscriptions.user_id → users.id | connections.(from_user, to_user) → users.id

3. Fonctionnalités implémentées et choix techniques

3.1 Authentification

- Inscription avec choix du rôle (membre ou hôte)
- Connexion avec vérification par password_verify() (bcrypt)
- Sessions PHP sécurisées avec session_regenerate_id() contre la fixation de session
- Redirection automatique selon le rôle après connexion
- Protection de toutes les pages sensibles via require_login() et require_host()

3.2 Espaces de co-working

- Listing avec filtres dynamiques : ville, type, prix maximum, équipements
- Requêtes SQL paramétrées construites dynamiquement avec PDO
- Page de détail : galerie, onglets description/équipements/avis, widget de réservation
- Statut de disponibilité en temps réel (available / limited / full)

3.3 Réservations

- Vérification de conflits de dates avant insertion en base
- Calcul automatique du prix total avec frais de service (9%)
- Statuts de réservation : pending, confirmed, cancelled
- L'hôte peut approuver ou annuler les demandes depuis son tableau de bord

3.4 Tableaux de bord

- Membre : statistiques (réservations actives, heures enregistrées), historique, abonnement

- Hôte : KPIs (revenus du mois, taux d'occupation), graphique des revenus par mois, gestion des annonces

3.5 Messagerie

- Liste de conversations avec compteur de messages non lus
- Thread de messages avec distinction envoyé/reçu
- Marquage automatique comme lu lors de l'ouverture d'une conversation

3.6 Réseau membres

- Annuaire public filtrable par ville
- Statut de disponibilité (open / busy / offline)
- Demandes de connexion entre membres

3.7 Avis

- Seuls les membres ayant une réservation confirmée peuvent laisser un avis
- Un avis par utilisateur par espace
- Mise à jour automatique de la note moyenne de l'espace après chaque avis

3.8 Architecture API (backend → frontend)

Tous les échanges de données entre PHP et JavaScript passent par des endpoints JSON. Chaque réponse contient un champ success (true/false) et un champ data ou message.

Méthode	Endpoint	Fonction
POST	/api/register.php	Création de compte
POST	/api/login.php	Authentification
GET	/api/logout.php	Déconnexion + redirection
GET	/api/spaces.php	Liste filtrée des espaces
GET	/api/space.php?id=	Détail d'un espace + dates réservées
POST	/api/book.php	Créer une réservation
GET	/api/dashboard.php	Données tableau de bord membre
GET/POST	/api/manager.php	Données hôte +

		approve/cancel/CRUD
GET	/api/members.php	Annuaire membres
POST	/api/members.php	Demande de connexion
GET/POST	/api/messages.php	Conversations + envoi de message
POST	/api/reviews.php	Soumettre un avis

3.9 Choix techniques

- PDO avec requêtes préparées : protection contre les injections SQL sur tous les endpoints
- Architecture MVC légère : séparation pages (vue), api/ (contrôleur), includes/ (modèle)
- CSS custom properties : système de design cohérent avec variables (couleurs, typographie, espacement)
- Google Fonts : Playfair Display (titres) + DM Sans (corps) pour un rendu professionnel
- Unsplash : images haute qualité pour les espaces via URLs directes en base de données
- Git / GitHub : versioning collaboratif avec branches par membre

4. Répartition des tâches:

Le travail a été divisé strictement par couche technologique, permettant à chaque membre de travailler en parallèle sans conflits de fichiers.

Membre	Role	Responsabilités principale
Hakiri Salma	Frontend — HTML/CSS	Structure HTML de toutes les pages, CSS global, composants visuels, intégration de la charte graphique.
Chaari Ines	Frontend — JavaScript	Logique interactive : filtres, calendrier de réservation, graphiques, appels
Jemai Frarah	Backend — PHP/SQL	Schéma BDD, endpoints API, authentification, logique métier, déploiement

Organisation Git: <https://github.com/farahjemai03/NomadNest.git>

- Dépôt partagé sur GitHub avec un branch par membre
- Pull Requests pour merger les changements vers main
- Convention de commit : courte description de la fonctionnalité ajoutée
- Fichiers .gitignore pour exclure les fichiers sensibles et temporaires

Points de coordination

- Membre 3 a fourni db.php et auth_check.php en premier pour débloquer les autres
- Accord préalable sur la structure JSON de chaque endpoint avant le développement JS
- Membre 1 a utilisé des id et data-attributes standardisés pour que le JS puisse s'y accrocher

5. Difficultés rencontrées et solutions apportées :

Merge conflicts Git Des conflits sont apparus lors des fusions de branches. Solution : définir clairement les dossiers de chaque membre (css/, js/, api/) pour éviter les modifications simultanées des mêmes fichiers. • Images Unsplash non chargées Certaines URLs Unsplash retournaient des erreurs selon la connexion réseau. Solution : tester plusieurs formats d'URL (?w=600&q;=80 vs auto=format&fit;=crop) et mettre à jour les images via HeidiSQL. • Structure PHP/HTML séparée Les fichiers HTML créés indépendamment devaient être fusionnés avec les shells PHP. Solution : copier le contenu HTML entre les balises dans les fichiers .php en conservant le PHP en en-tête. • Sessions et redirections Les pages protégées redirigeaient incorrectement car session_start() n'était pas appelé avant require_once auth_check.php. Solution : standardiser l'ordre : session_start() → db.php → auth_check.php. • Requêtes SQL dynamiques avec filtres Construire des requêtes WHERE variables avec PDO sans injection SQL. Solution : construire les clauses WHERE dans un tableau \$where et \$params, puis les assembler avant prepare(). • Unfinished Git merge Une fusion interrompue a bloqué git pull. Solution : git merge --abort puis git pull origin main --no-edit.

6. Perspectives d'amélioration et fonctionnalités envisagées mais non réalisées:

Perspectives d'amélioration Plusieurs fonctionnalités ont été conçues mais non implémentées par manque de temps. Elles constituent des pistes d'amélioration prioritaires pour une version future :

Fonctionnalité	Description
Paiement en ligne	Intégration d'une passerelle de paiement (Stripe) pour les réservations et abonnements.

Messagerie temps réel	Remplacement du polling par WebSockets (Ratchet PHP) pour une messagerie instantanée.
Notifications	Notifications in-app et par email (SendGrid) pour les nouvelles réservations et messages.
Carte interactive	Affichage des espaces sur une carte Leaflet.js avec clustering par ville.
Upload d'images	Permettre aux hôtes d'uploader leurs propres photos via une gestion de fichiers PHP.
Application mobile	Version Progressive Web App (PWA) avec mode hors-ligne pour les nomades.
Système de favori	Permettre aux membres de sauvegarder des espaces dans une liste de favoris.
Calendrier hôte	Vue calendrier pour les hôtes montrant toutes les réservations par espace.
Abonnements payants	Implémenter les niveaux Pro et Business avec des avantages réels (réservations prioritaires)
Recherche full-text	Moteur de recherche par mot-clé sur le nom, la description et la ville des espaces.